

Upgrade Path

Guide complet : Revue de code, Déploiement Docker/Portainer, HTTPS et Maintenance.

À propos

Upgrade Path est une application d'intelligence de mise à jour FortiOS pour les ingénieurs réseau. Ce guide couvre l'ensemble du cycle de vie : revue de code, déploiement Docker/Portainer, configuration HTTPS et maintenance.

Sommaire

Upgrade Path -- Guide de Déploiement

Sommaire

1. Revue de code
2. Architecture du projet
3. Déploiement Docker / Portainer
4. HTTPS -- Installation du certificat TLS
5. Mise à jour / Maintenance
6. Dépannage
7. Checklist finale

Note

Guide complet : Revue de code, Déploiement Docker/Portainer, HTTPS et Maintenance.

Ce guide explique pas à pas comment déployer l'application Upgrade Path sur une VM interne Docker administrée par Portainer Community Edition. L'application sera accessible en HTTPS sur le réseau interne à l'adresse <https://upgrade-path.sns-security.lan>.

Note

■ Déploiement interne uniquement. L'application ne sera PAS exposée sur Internet. Certificat TLS : PKI entreprise. Nom de domaine : upgrade-path.sns-security.lan. Deux services : web (interface + API) et scheduler (collectes planifiées 07:00 et 15:30 Europe/Paris).

■ Machine	■ Où ?	■ Qui ?	■ Accès
SOURCE (dev)	VPS IONOS	Da Vinci (Hermes)	Automatique (CI/CD)
DESTINATION (prod)	VM Entreprise	Valentin	Console/SSH (root)

Sommaire

1. Revue de code
2. Architecture du projet
3. Déploiement Docker / Portainer
4. HTTPS -- Installation du certificat TLS
5. Mise à jour / Maintenance
6. Dépannage
7. Checklist finale

1. Revue de code

Upgrade Path est une application Python d'intelligence de mise à jour FortiOS. Elle analyse les firmwares, vulnérabilités CVE, matrices de compatibilité, et prépare les chemins de montée de version pour les ingénieurs réseau.

Deux conteneurs Docker : web (interface web + API) et scheduler (collectes planifiées), partageant la même image `fortios-upgrade-intelligence:local`.

Note

■ L'image Docker est construite localement puis importée dans Portainer. Image : `fortios-upgrade-intelligence:local`.

Corrections appliquées (revue du 3 août 2026)

■ Problème	■ Correction	■ Fichier(s)
CI rouge : actions/checkout@v7 n'existe pas	Rétrogradé vers checkout@v4, setup-python@v5	<code>`.github/workflows/tests.yml`</code>
cert_direct_install = False quand FORTIOS_CERT_HELPER_SOCKET défini	Ajout <code>`bool(os.environ.get(...))`</code> dans la condition	<code>`scripts/fortios_server.py`</code>
Push SSH échoue (pas de clé en conteneur)	Fallback HTTPS avec GITHUB_TOKEN depuis .env	workflow git push
Travaux parallèles bloqués (worktrees morts)	Nettoyage automatique <code>.worktrees/t_*</code> après chaque cycle	maintenance
Tests e2e : 403 au lieu de 409 sur install sans ticket	Correction certificat direct_install (voir ci-dessus)	<code>`tests/test_api.py`</code>

Détails techniques :

- Serveur Python custom HTTP(S) sur port interne 8000
- Listener unique : HTTP ou HTTPS, jamais les deux en parallèle
- Healthcheck intégré
- CLI `fortios-certctl` pour la gestion des certificats
- CI : `ruff` + `python-tests` + `e2e-tests` + `docker-build-push` + `security-scan` (Trivy)

2. Architecture du projet

```

upgrade_path_forti/
├── app/                # Application web (frontend JS)
├── scripts/            # Serveur Python, CLI certificat
│   ├── fortios_server.py # Serveur HTTP/HTTPS principal
│   └── fortios_certctl   # CLI de gestion des certificats
├── tests/              # Tests unitaires + e2e
├── docker/             # Configuration Docker
├── deploy/             # Scripts de déploiement
├── data/               # Données métier (catalogue, CVE, etc.)
├── docs/               # Documentation et tutoriels
├── docker-compose.portainer-import.yml # Stack Portainer (image locale)
├── docker-compose.yml  # Stack Docker Compose standard
├── Dockerfile          # Construction de l'image
└── .github/workflows/  # CI/CD GitHub Actions

```

Volumes persistants (créés par la Stack Portainer) :

- `fortios-data` : données métier (catalogue firmware, CVE, compatibilités)
- `fortios-docs` : rapports et documents générés
- `fortios-certificates` : certificats TLS normalisés

Note

■ Ne jamais supprimer ces volumes pendant une mise à jour ou une recréation de Stack.

Déploiement

3. Déploiement Docker / Portainer — déploiement via image Docker locale dans Portainer

Ce que tu vas faire

Déployer l'application sur une VM interne via Portainer Community Edition en important une image Docker locale, puis créer la Stack avec deux services (web + scheduler).

Pré-requis

- Une VM interne avec Docker et Portainer Community Edition fonctionnels
- Le dépôt `upgrade_path_forti` dans sa version validée sur la machine source
- Un poste ouvrant l'interface web Portainer
- Un accès LAN à la VM interne (firewall TCP/8000 puis TCP/443)

Fichiers nécessaires

Fichier	Origine	Utilisation
<code>`fortios-upgrade-intelligence.tar`</code>	Exporté depuis la VM source (<code>`docker save`</code>)	Import de l'image dans Portainer
<code>`docker-compose.portainer-import.yml`</code>	Dépôt Git	Import de la Stack dans Portainer

ÉTAPE 1 -- Construire l'image sur la VM source

```
cd /chemin/vers/upgrade_path
git status --short # doit être vide ou contenir uniquement des fichiers non critiques
docker build --pull -t fortios-upgrade-intelligence:local .
docker image inspect fortios-upgrade-intelligence:local --format '{{.Id}} {{.Created}}'
```

ÉTAPE 2 -- Exporter l'image au format TAR

```
docker save -o ~/fortios-upgrade-intelligence.tar fortios-upgrade-intelligence:local
sha256sum ~/fortios-upgrade-intelligence.tar > ~/fortios-upgrade-intelligence.tar.sha256
ls -lh ~/fortios-upgrade-intelligence.tar*
```

Note

■ Garder l'archive au format `.tar`. Ne pas la compresser en `.tar.gz` : l'action Import de Portainer attend l'archive produite par `docker save`.

Transférer `fortios-upgrade-intelligence.tar` et `docker-compose.portainer-import.yml` sur le poste qui ouvre Portainer.

ÉTAPE 3 -- Importer l'image dans Portainer

1. Ouvrir Portainer, sélectionner l'environnement Docker cible
2. Menu latéral → Images (pas Registries)
3. Cliquer Import
4. Sélectionner `fortios-upgrade-intelligence.tar`, confirmer
5. Vérifier que `fortios-upgrade-intelligence:local` apparaît dans la liste

ÉTAPE 4 -- Déployer la Stack

1. Menu latéral → Stacks → Add stack
2. Nom : `upgrade-path`
3. Méthode : Upload → sélectionner `docker-compose.portainer-import.yml`
4. Section Environment variables :

```
PUID=1000
PGID=1000
FORTIOS_HTTP_BIND_ADDRESS=0.0.0.0
FORTIOS_HTTP_PORT=8000
FORTIOS_TLS_CERT=
FORTIOS_TLS_KEY=
FORTIOS_TLS_HOSTNAME=
FORTIOS_RUN_ON_START=0
FORTIOS_EMAIL_ENABLED=false
FORTIOS_APP_URL=http://IP_LAN_VM:8000/app/
```

Remplacer `IP_LAN_VM` par l'adresse IP réelle de la VM. `PUID` et `PGID` strictement supérieurs à zéro.

1. Cliquer Deploy the stack

La Stack crée deux services (`web`, `scheduler`) et trois volumes persistants (`fortios-data`, `fortios-docs`, `fortios-certificates`).

Note

■ Conserver `FORTIOS_EMAIL_ENABLED=false` pour le déploiement initial. Ajouter les variables SMTP uniquement si les notifications doivent être activées.

ÉTAPE 5 -- Vérifier le fonctionnement HTTP

Dans Portainer :

- Containers → `web` et `scheduler` doivent être running (healthy)
- Logs de `web` : doit annoncer HTTP sur le listener interne 8000
- Logs de `scheduler` : doit annoncer les prochaines collectes (07:00 et 15:30 Europe/Paris)

Depuis le réseau interne :

```
http://IP_LAN_VM:8000/app/
```

Depuis la VM Docker (SSH) :

```
WEB_CONTAINER="$(docker ps --filter label=com.docker.compose.project=upgrade-path --filter label=com
docker logs --tail 50 "$WEB_CONTAINER"
docker inspect --format '{{.State.Health.Status}}' "$WEB_CONTAINER"
# Doit afficher : healthy
```

Note

- Le port 8000 doit être accessible uniquement depuis les sous-réseaux internes autorisés. Ne jamais le publier sur Internet. HTTP et HTTPS ne sont jamais exposés en parallèle.

- Note
- Le port 8000 doit être accessible uniquement depuis les sous-réseaux internes autorisés. Ne jamais le publier sur Internet. HTTP et HTTPS ne sont jamais exposés en parallèle.

4. HTTPS -- Installation du certificat TLS

Ce que tu vas faire

Installer un certificat TLS de la PKI interne pour que l'application soit accessible en HTTPS sur <https://upgrade-path.sns-security.lan>.

Contexte

L'URL finale sera : <https://upgrade-path.sns-security.lan>

Certificat fourni par la PKI interne de l'entreprise (AD CS, EJBCA, etc.). Pas de Let's Encrypt -- le suffixe .lan n'est pas signé par les AC publiques. Le certificat doit contenir dans ses SAN DNS :
upgrade-path.sns-security.lan (ou wildcard *.sns-security.lan).

Option A -- PKI interne (PFX/P12)

Obtenir le certificat

Demander à l'équipe IT un fichier PFX/P12 pour `upgrade-path.sns-security.lan` (ou wildcard `*.sns-security.lan`).

Transférer vers la VM

```
scp serveur-interne.pfx root@VM_IP:/tmp/
```

Retrouver le conteneur web

```
FQDN='upgrade-path.sns-security.lan'
WEB_CONTAINER="$(docker ps --filter label=com.docker.compose.project=upgrade-path --filter label=com
test -n "$WEB_CONTAINER"
```

Installer le certificat (PFX avec mot de passe)

```
docker cp /tmp/serveur-interne.pfx "$WEB_CONTAINER":/tmp/certificat.pfx
read -rsp 'Mot de passe PFX : ' PFX_PASSWORD; echo
printf '%s' "$PFX_PASSWORD" | docker exec -i "$WEB_CONTAINER" sh -c 'umask 077; cat > /tmp/cert-password'
unset PFX_PASSWORD

docker exec "$WEB_CONTAINER" fortios-certctl install \
    /tmp/certificat.pfx \
    --password-file /tmp/cert-password \
    --hostname "$FQDN" \
    --output-dir /opt/fortios/certificates/active
INSTALL_RC=$?

docker exec "$WEB_CONTAINER" rm -f /tmp/certificat.pfx /tmp/cert-password
test "$INSTALL_RC" -eq 0
```

Installer le certificat (PFX sans mot de passe)

```
docker exec "$WEB_CONTAINER" fortios-certctl install \
  /tmp/certificat.pfx \
  --hostname "$FQDN" \
  --output-dir /opt/fortios/certificates/active
INSTALL_RC=$?
docker exec "$WEB_CONTAINER" rm -f /tmp/certificat.pfx
test "$INSTALL_RC" -eq 0
```

Option B -- Certificat, clé et chaîne séparés

Transférer les fichiers

```
scp serveur.crt serveur.key chaine.p7b root@VM_IP:/tmp/
```

Copier dans le conteneur

```
docker cp /tmp/serveur.crt "$WEB_CONTAINER":/tmp/serveur.crt
docker cp /tmp/serveur.key "$WEB_CONTAINER":/tmp/serveur.key
docker cp /tmp/chaine.p7b "$WEB_CONTAINER":/tmp/chaine.p7b
```

Installer

```
docker exec "$WEB_CONTAINER" fortios-certctl install \
  /tmp/serveur.crt \
  --key /tmp/serveur.key \
  --chain /tmp/chaine.p7b \
  --hostname "$FQDN" \
  --output-dir /opt/fortios/certificates/active
INSTALL_RC=$?

docker exec "$WEB_CONTAINER" rm -f /tmp/serveur.crt /tmp/serveur.key /tmp/chaine.p7b
test "$INSTALL_RC" -eq 0
```

Note

■ Si aucune chaîne séparée n'est nécessaire, omettre l'option --chain. Si la clé est chiffrée, utiliser --password-file /tmp/cert-password comme dans l'Option A.

Vérifier le certificat installé

```
docker exec "$WEB_CONTAINER" openssl x509 \
  -in /opt/fortios/certificates/active/fullchain.pem \
  -noout -subject -issuer -dates \
  -ext subjectAltName,extendedKeyUsage
```

Activer HTTPS dans Portainer

1. Stacks → upgrade-path → Editor
2. Modifier les variables d'environnement :

```
FORTIOS_HTTP_PORT=443
FORTIOS_TLS_CERT=/opt/fortios/certificates/active/fullchain.pem
FORTIOS_TLS_KEY=/opt/fortios/certificates/active/privkey.pem
FORTIOS_TLS_HOSTNAME=upgrade-path.sns-security.lan
FORTIOS_APP_URL=https://upgrade-path.sns-security.lan/app/
```


1. Cliquer Update the stack (ne pas re-pull, l'image est locale)
2. Le port hôte 443 est maintenant mappé vers le listener interne 8000 en TLS

Vérifier HTTPS

Depuis un poste approuvé :

```
curl --fail --silent --show-error --cacert /chemin/ca-interne.pem \
  "https://upgrade-path.sns-security.lan/app/" >/dev/null
```

Navigateur :

```
https://upgrade-path.sns-security.lan/app/
```

Note

■ Le poste doit faire confiance à la CA interne et résoudre le FQDN. Si le DNS n'est pas encore en place, ajouter `IP_VM upgrade-path.sns-security.lan` dans `/etc/hosts`.

Maintenance

5. Mise à jour / Maintenance — procédure de montée de version et renouvellement

Mise à jour de l'application

1. Sur la machine source, récupérer la dernière version : `git pull`
2. Reconstruire l'image : `docker build --pull -t fortios-upgrade-intelligence:local .`
3. Exporter : `docker save -o ~/fortios-upgrade-intelligence.tar fortios-upgrade-intelligence:local`
4. Transférer le `.tar` sur le poste Portainer
5. Dans Portainer → Images → Import → sélectionner le nouveau `.tar`
6. Stacks → `upgrade-path` → Update the stack (sans re-pull)

Renouvellement du certificat

1. Conserver la Stack en HTTPS
2. Répéter la méthode PFX ou certificat/clé/chaîne avec les nouveaux fichiers
3. Dans Portainer, Containers → redémarrer uniquement le conteneur `web`
4. Ne pas redémarrer le scheduler, ne pas supprimer les volumes

Note

■ Le CLI `fortios-certctl` valide entièrement la nouvelle génération avant activation. Si l'installation échoue, le certificat actif est conservé.

Retour temporaire en HTTP (exceptionnel)

Dans les variables de la Stack, définir :

```
FORTIOS_HTTP_PORT=8000
FORTIOS_TLS_CERT=
FORTIOS_TLS_KEY=
FORTIOS_TLS_HOSTNAME=
FORTIOS_APP_URL=http://IP_LAN_VM:8000/app/
```

Puis Update the stack. Il n'existe volontairement aucun mode dans lequel HTTP et HTTPS sont servis simultanément.

6. Dépannage

■ Problème	■ Vérification / Action
Stack ne se déploie pas	Vérifier toutes les variables d'environnement requises
Conteneur web pas healthy	<code>`docker logs --tail 100 "\$WEB_CONTAINER"`,</code> vérifier les variables TLS
Le CLI refuse le certificat	SAN DNS présent ? Correspondance SAN/FQDN ? Période de validité ? Usage TLS serveur ?
Navigateur affiche une alerte	Résolution DNS, SAN, dates, chaîne, CA interne dans le magasin de confiance
HTTP/8000 toujours accessible après activation HTTPS	Vérifier <code>`FORTIOS_HTTP_PORT=443`</code> , mettre à jour la Stack, pas d'ancien conteneur
Portainer : <code>`no such image`</code>	Importer l'image locale (<code>`docker save`</code> → Import), ne pas utiliser de registre
Scheduler ne collecte pas	Vérifier les logs, fuseau horaire Europe/Paris, <code>`FORTIOS_RUN_ON_START`</code>
Données perdues après redéploiement	Ne pas supprimer les volumes <code>`fortios-data`</code> et <code>`fortios-docs`</code>
Variables TLS incohérentes	<code>`FORTIOS_TLS_CERT`</code> , <code>`FORTIOS_TLS_KEY`</code> et <code>`FORTIOS_TLS_HOSTNAME`</code> doivent être définis ensemble

7. Checklist finale

- [] Image `fortios-upgrade-intelligence:local` présente dans Portainer
- [] Stack `upgrade-path` contient uniquement `web` et `scheduler`
- [] Trois volumes persistants existent (`fortios-data`, `fortios-docs`, `fortios-certificates`)
- [] Conteneur `web` est healthy
- [] `http://IP_LAN_VM:8000/app/` accessible depuis le LAN
- [] Certificat TLS installé dans `/opt/fortios/certificates/active/`

- [] SAN DNS correct (`upgrade-path.sns-security.lan`)
- [] DNS interne résout le FQDN
- [] CA interne approuvée sur les postes clients
- [] `https://upgrade-path.sns-security.lan/app/` accessible, cadenas vert
- [] HTTP/8000 n'est plus accessible après activation HTTPS
- [] Firewall limite TCP/443 aux réseaux internes autorisés
- [] Aucun secret ou certificat privé dans Git ou dans l'image
- [] Ancienne instance arrêtée uniquement après validation complète

■ Déploiement terminé. Application accessible en HTTPS sur le réseau interne. ■

■ Ressources

- Repo : https://github.com/Tetrax/upgrade_path_forti
- Image : `fortios-upgrade-intelligence:local` (build local, import Portainer)
- CI : https://github.com/Tetrax/upgrade_path_forti/actions